مقدمة في CSS وشرح أهم أفكارها

بحث أو تقرير دراسي

الخانة القابلة للتعبئة	البيان
_____مريم هيثم جمعه فرج_	الاسم
_____فرقه تانيه___	الصف/الفرقة
_____تطبيقات الانترنت____	المادة
_____د مها زعلول___	اسم المعلم/المعلمة
الدلتا العالي لنظم المعلومات الاداريه و__ _____المحاسبيه	المدرسة/الجامعة
_____د____	المجموعه
_____2500246__	الكود

ملاحظة: تُترك الخانات السابقة فارغة لتعبئتها لاحقًا، ولا يتضمن هذا التقرير أي بيانات شخصية مفترضة.

1. مقدمة عن CSS

ما المقصود بـ CSS؟

CSS هي اختصار لعبارة Cascading Style Sheets، وهي لغة تُستخدم لوصف .cascading أي أوراق الأنماط المت.
شكل صفحات الويب وطريقة عرض عناصرها، مثل الألوان والخطوط والمسافات والأحجام ومواقع العناصر.

تساعد CSS على فصل المحتوى عن التصميم؛ فتتولى HTML بناء هيكل الصفحة، بينما تتولى CSS تحديد مظهر هذا الهيكل. وهذا الفصل يجعل تعديل التصميم أسهل، ويتيح تطبيق النمط نفسه على صفحات متعددة.

لماذا CSS مهمة؟

- تحسن سهولة القراءة وتجعل الصفحة أكثر وضوحًا.
- تمنح المصمم تحكمًا في الألوان والخطوط والتخطيط.
- تساعد على إنشاء تصميم متجاوب مع الهاتف والحاسوب.
- تقلل تكرار التنسيقات عند استخدام ملفات خارجية.

الفرق بين HTML وCSS وJavaScript

المهمة الأساسية	التقنية
بناء محتوى الصفحة وهيكلها، مثل العناوين والفقرات والصور.	HTML
تنسيق المحتوى والتحكم في مظهره وتخطيطه واستجابته.	CSS
إضافة السلوك والتفاعل، مثل القوائم المنسدلة والتحقق من النماذج.	JavaScript

---

2. كيف تعمل CSS؟

تعمل CSS من خلال قواعد تحدد العناصر التي نريد تنسيقها، ثم تحدد الخاصية المراد تغييرها والقيمة التي ستأخذها هذه الخاصية. تتكون القاعدة عادةً من Selector، ثم مجموعة من التصريحات داخل أقواس معقوفة.

المفهوم	المعنى
Selector	المحدد الذي يختار عنصرًا أو مجموعة عناصر، مثل p أو .card.
Property	الخاصية التي نريد تغييرها، مثل color أو font-size.
Value	القيمة التي نمنحها للخاصية، مثل blue أو 16px.

```
.notice {
  color: #1d4ed8;
  background-color: #eff6ff;
  padding: 12px;
}
```

في المثال السابق يحدد `.notice` العناصر التي تحمل class بهذا الاسم، ثم يغير لون النص والخلفية ويضيف حشوًا داخليًا.

عند تطبيق القاعدة، يبحث المتصفح عن العناصر المطابقة للمحدد، ثم يطبق التصريحات الصحيحة عليها. وإذا تعارضت قواعد متعددة، تستخدم CSS آليات مثل cascade وspecificity وinheritance لتحديد النتيجة النهائية.

3. طرق إضافة CSS إلى صفحات HTML

يمكن إضافة CSS إلى HTML بثلاث طرق رئيسية. يختلف الاختيار حسب حجم المشروع وعدد الصفحات وسهولة الصيانة المطلوبة.

أولًا: Inline CSS

تُكتب القاعدة داخل الخاصية style في العنصر نفسه، وهي مناسبة لتجربة سريعة أو تعديل محدود، لكنها تصبح صعبة الإدارة عند كثرة العناصر.

```
<p style="color: blue; font-size: 18px;">فقرة منسقة</p>
```

ثانيًا: Internal CSS

تُكتب القواعد داخل وسم <style> في صفحة HTML، وتفيد عندما يكون التنسيق خاصًا بصفحة واحدة.

```
<style>
  h1 { color: darkgreen; }
</style>
```

ثالثًا: External CSS

يُحفظ التنسيق في ملف مستقل، مثل styles.css، ثم يُربط بصفحة HTML باستخدام وسم link.

```
<link rel="stylesheet" href="styles.css">
```

```
/* داخل ملف styles.css */
body {
  font-family: Arial, sans-serif;
  background: #f8fafc;
}
```

الأفضلية العملية: يُفضل استخدام External CSS في المشاريع الحقيقية؛ لأنه يعيد استخدام التنسيق في صفحات متعددة، ويسهل تنظيمه وصيانته، وقد يستفيد المتصفح من تخزين الملف مؤقتًا.

---

4. أهم أنواع المحددات Selectors

المحدد هو الجزء الذي يخبر المتصفح بالعناصر التي ستتلقى التنسيق. اختيار المحدد المناسب يجعل القواعد أكثر وضوحًا وقابلية لإعادة الاستخدام.

ما الذي يحدده؟	مثال CSS	النوع
كل عناصر الفقرة p.	`p { color: #333; }`	محدد العنصر
كل عنصر يحمل class="note".	`.note { padding: 8px; }`	class
العنصر الذي يحمل id="main-title".	`#main-title { font-size: 32px; }`	id
جميع عناصر الصفحة.	`* { box-sizing: border-box; }`	المحدد العام
فقرات p الموجودة داخل عنصر card.	`.card p { line-height: 1.6; }`	المحدد المتداخل

عناصر input التي تحمل النوع email.	`input[type="email"] { border-color: teal; }`	محدد attribute

من الأفضل استخدام class للتنسيقات المتكررة، واستخدام id للعناصر الفريدة عند الحاجة، مع تجنب الاعتماد على محددات شديدة التعقيد حتى تظل القواعد سهلة الفهم.

5. أهم الأفكار والمفاهيم

cascade وspecificity وinheritance

تعني cascade أن المتصفح يجمع القواعد المناسبة ثم يختار القاعدة النهائية وفق مصدرها وترتيبها ودرجة specificity. وعند تساوي القوة، تنتصر القاعدة التي تظهر لاحقًا في الملف.

تشير specificity إلى مدى تحديد القاعدة للعنصر. فعادةً يكون id أقوى من class، ويكون class أقوى من محدد العنصر. أما inheritance فتعني أن بعض الخصائص، مثل لون النص ونوع الخط، يمكن أن تنتقل من العنصر الأب إلى العناصر الأبناء ما لم تُحدد لها قيمة مختلفة.

نصيحة: ابدأ بمحددات بسيطة، واستخدم class بأسماء واضحة بدلًا من رفع specificity باستمرار أو اللجوء إلى !important دون سبب ضروري.

Box Model

كل عنصر في الصفحة يُعامل كصندوق يتكون من content ثم padding ثم border ثم margin. يؤثر فهم هذه الطبقات في ضبط الحجم والمسافات وتجنب تمدد العناصر بطريقة غير متوقعة.

الجزء	وظيفته
Content	المحتوى الداخلي، مثل النص أو الصورة.
Padding	مسافة داخلية بين المحتوى والحدود.
Border	الإطار المحيط بالعنصر.
Margin	مسافة خارجية تفصل العنصر عن غيره.

```
* {
  box-sizing: border-box;
}
.box {
  width: 280px;
  padding: 16px;
  border: 2px solid #2563eb;
  margin: 20px;
}
```

الألوان والوحدات والخطوط

يمكن كتابة الألوان بأسماء جاهزة أو بصيغ مثل HEX وRGB وHSL. أما الوحدات فمنها وحدات ثابتة مثل px، ووحدات نسبية مثل % وem وrem، ووحدات مرتبطة بحجم الشاشة مثل vw وvh. وتفيد rem في إنشاء أحجام أكثر اتساقًا وقابلية للتكيف.

تتحكم خصائص مثل font-family وfont-size وfont-weight وline-height في شكل النص ووضوحه. من الأفضل اختيار خط سهل القراءة، وتحديد بدائل في font-family عند عدم توفر الخط الأساسي.

الهوامش والحشو والحدود والخلفيات

تستخدم margin للمسافة خارج العنصر، بينما يستخدم padding للمسافة داخله. وتحدد border سماكة الإطار ونمطه ولونه، أما background فتتحكم في لون الخلفية أو صورتها. ينبغي اختيار مسافات متناسقة وألوان ذات تباين كافٍ بين النص والخلفية.

الفرق بين display وposition

تحدد display طريقة مشاركة العنصر في التخطيط، مثل block وinline وflex وgrid وnone. أما position فتحدد طريقة تموضع العنصر، مثل static وrelative وabsolute وfixed وsticky.

الفكرة	مثال	الخاصية
تغيير نظام ترتيب العناصر داخل الحاوية.	display: flex;	display
تحديد طريقة ارتباط العنصر بمكانه في الصفحة.	position: fixed;	position

6. التخطيط Layout

Flexbox

نظام تخطيط مناسب لترتيب العناصر في اتجاه واحد: صف أو عمود. يفيد في محاذاة العناصر وتوزيع Flexbox المسافات بينها، مثل أزرار شريط التنقل أو عناصر بطاقة صغيرة.

```
.toolbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 12px;
}
```

في هذا المثال توضع العناصر في صف، وتوزع المسافات بينها، ثم تتم محاذاتها رأسيًا في المنتصف.

CSS Grid

نظام ثنائي الأبعاد يتعامل مع الصفوف والأعمدة معًا، ولذلك يناسب صفحات المنتجات أو لوحات المعلومات أو Grid التخطيطات التي تحتاج إلى شبكة واضحة.

```
.gallery {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
}
```

النظام	متى يُستخدم؟
Flexbox	عندما يكون الترتيب الأساسي في اتجاه واحد، مثل صف أزرار أو قائمة أفقية.
Grid	عندما نحتاج إلى صفوف وأعمدة، مثل شبكة بطاقات أو تخطيط صفحة كامل.

---

7. التصميم المتجاوب Responsive Design

التصميم المتجاوب يجعل الصفحة مناسبة لأحجام الشاشات المختلفة. ويعتمد على وحدات مرنة، وصور قابلة للتقلص، وmedia queries التي تطبق قواعد إضافية عند تحقق شرط معين.

يبدأ أسلوب mobile-first بتصميم القواعد الأساسية للهواتف أولًا، ثم إضافة تحسينات للشاشات الأكبر. يساعد هذا الأسلوب على التركيز على المحتوى الأساسي وتقليل القواعد غير الضرورية.

```
.profile-card {
  width: 100%;
  padding: 16px;
```

```
  border-radius: 12px;
}

@media (min-width: 700px) {
  .profile-card {
    max-width: 520px;
    margin: 0 auto;
  }
}
```

تظهر البطاقة بعرض مرن على الهاتف، ثم يصبح لها عرض أقصى وتتمركز في الشاشة الأكبر. ويمكن إضافة نقاط توقف أخرى عند الحاجة، لكن يفضل إبقاء التصميم بسيطًا واختبار نقاط التوقف الفعلية بدل اختيار أرقام كثيرة عشوائيًا.

8. أمثلة تطبيقية متدرجة

الأمثلة الآتية تنتقل من تنسيق بسيط إلى تخطيط أكثر تنظيمًا. يمكن تجربة كل مثال في ملف HTML مع ملف CSS خارجي لتثبيت المفاهيم عمليًا.

المثال الأول: تنسيق عنوان وفقرة

HTML المختصر

```
<h1 class="title">تعلم CSS</h1>
<p class="intro">CSS تجعل صفحات الويب أوضح وأجمل.</p>
```

CSS

```
.title { color: #1d4ed8; }
.intro { line-height: 1.7; color: #475569; }
```

تظهر النتيجة بعنوان أزرق وفقرة بلون هادئ ومسافة أسطر مريحة للقراءة.

المثال الثاني: إنشاء زر

HTML المختصر

```
<button class="btn">ابدأ الآن</button>
```

CSS

```
.btn {
  background: #2563eb;
  color: white;
```

```
  border: 0;
  padding: 10px 18px;
  border-radius: 8px;
}
.btn:hover { background: #1d4ed8; }
```

يظهر الزر بخلفية زرقاء وحواف مستديرة، ويتغير لونه قليلًا عند مرور المؤشر فوقه.

المثال الثالث: بطاقة Profile

HTML المختصر

```
<article class="profile">
  <h2>سارة أحمد</h2>
  <p>طالبة مهتمة بتصميم الويب.</p>
</article>
```

CSS

```
.profile {
  max-width: 300px;
  padding: 20px;
  background: #f8fafc;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
}
.profile h2 { margin-top: 0; }
```

تظهر البطاقة كصندوق مستقل بعرض محدود وحشو داخلي وحدود وحواف مستديرة، مع إزالة المسافة العلوية الزائدة من العنوان.

المثال الرابع: قائمة تنقل بسيطة

HTML المختصر

```
<nav class="nav">
  <a href="#">الرئيسية</a>
  <a href="#">من نحن</a>
  <a href="#">تواصل</a>
</nav>
```

CSS

```
.nav {
  display: flex;
```

```
  gap: 20px;
  padding: 12px;
}
.nav a {
  color: #0f172a;
  text-decoration: none;
}
.nav a:hover { color: #2563eb; }
```

ترتب Flexbox الروابط في صف واحد مع مسافات بينها، وتغير قاعدة hover لون الرابط عند التفاعل معه.

المثال الخامس: تخطيط أعمدة باستخدام Grid

HTML المختصر

```
<section class="columns">
  <div>المحتوى الأول</div>
  <div>المحتوى الثاني</div>
  <div>المحتوى الثالث</div>
</section>
```

CSS

```
.columns {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
}
.columns > div {
  padding: 16px;
  background: #e0f2fe;
}
```

تنشئ Grid ثلاثة أعمدة متساوية، وتضيف مسافة بينها وخلفية خفيفة لكل عمود. ويمكن جعلها عمودًا واحدًا في الشاشات الصغيرة باستخدام media query.

9. أهم النصائح والأخطاء الشائعة

يمكن استخدام القائمة الآتية كمرجع سريع عند كتابة CSS أو مراجعة مشروع صغير:

- ☐ **تنظيم الملفات** — افصل HTML وCSS وJavaScript، وضع الملفات في مجلدات واضحة بأسماء مفهومة.
- ☐ **تسمية class** — اختر أسماء تصف الوظيفة أو المحتوى، مثل card وprimary-button، وتجنب الأسماء المرتبطة بلون مؤقت فقط.
- ☐ **تجنب !important** — لا تستخدمه إلا عند وجود سبب واضح؛ لأنه يصعّب تتبع التعارضات وتعديل القواعد لاحقًا.
- ☐ **التحقق من التباين** — تأكد من أن لون النص واضح أمام الخلفية، خصوصًا في العناوين والأزرار والنصوص

الصغيرة.

- [ ] **اختبار الشاشات** — جرّب التصميم على الهاتف والجهاز اللوحي والحاسوب، وبأحجام مختلفة للنافذة.
- [ ] **إعادة الاستخدام** — اجمع القواعد المشتركة في classes قابلة لإعادة الاستخدام بدل نسخ التنسيق لكل عنصر.
- [ ] **مراجعة التخطيط** — استخدم أدوات المطور في المتصفح لفحص Box Model ومعرفة سبب المسافات أو التعارضات.

---

10. خاتمة وخطوات لاحقة

تعرفنا في هذا التقرير إلى CSS ودورها في تنسيق صفحات الويب، وفهمنا الفرق بينها وبين HTML وJavaScript، وطريقة عمل selectors وproperties وvalues. كما استعرضنا طرق إضافة CSS، وأهم المحددات والمفاهيم مثل cascade وspecificity وinheritance وBox Model، ثم طبقنا Flexbox وGrid وmedia queries في أمثلة عملية.

للتدرب بعد ذلك، يمكن إنشاء صفحة شخصية صغيرة، ثم تحسينها تدريجيًا بإضافة بطاقة Profile وقائمة تنقل وتخطيط متجاوب. ومن المفيد أيضًا إعادة كتابة الأمثلة السابقة بألوان ووحدات مختلفة، وفحص النتيجة باستخدام أدوات المطور، ثم تجربة بناء مشروع بسيط من عدة صفحات بملف CSS خارجي مشترك.

الخلاصة: ابدأ بهيكل HTML واضح، ثم أضف CSS بسيطة ومنظمة، واختبر النتيجة باستمرار على أكثر من شاشة. الممارسة المتدرجة أهم من حفظ الخصائص دون تطبيق.

---

11. المراجع والمصادر المعتمدة

استُخدمت المصادر الآتية بوصفها مراجع تعليمية موثوقة لفهم أساسيات CSS وخصائصها وتخطيط الصفحات والتصميم المتجاوب:

ما تم الاستفادة منه	الرابط	المصدر
مرجع شامل لخصائص CSS والمحددات والقيم وأمثلة الاستخدام.	developer.mozilla.org — CSS	MDN Web Docs — CSS
معلومات عن معايير CSS وتطورها والجهة المسؤولة عن مواصفات	w3.org — CSS	W3C CSS

الويب.		
دروس تعليمية مبسطة حول التخطيط وBox Model والتصميم المتجاوب.	web.dev — Learn CSS	web.dev Learn CSS
شروحات وأدلة تعليمية عملية حول Flexbox وGrid وتقنيات CSS الشائعة.	css-tricks.com — Guides	CSS-Tricks Guides